**Centro Federal de Educação Tecnológica de Minas Gerais**
Professora: Poliana Corrêa

# RELATÓRIO PRÁTICA 4 - LAOC
# Blocos lógicos, ULA e Memória de dados

ALUNOS:
GIOVANNI ESTEVAM DOS SANTOS
JOÃO VITOR LOPES FAGUNDES

**Belo Horizonte - MG**
**2025**

# 1. Introdução

A prática 4 ( Blocos lógicos, ULA e Memória de dados ), teve como foco principal construir uma ULA e mais ao fim construir uma memória de dados. Para fazer a ULA é necessário fazer uma porta and, e or, um somador completo e o menor que (slt). Com isso feito é necessário fazer suas respectivas funções para 32 bits, já que primeiro fazemos em 1 bit. E então finalmente tratar o overflow e criar o extensor de sinal. Além de criação de uma memória de dados de 64 bytes.

# 2. Desenvolvimento

Nosso projeto cria uma Unidade Lógica e Aritmética (ULA) de 32 bits, totalmente funcional e integrada a um módulo de topo (**ULA_top**) que a conecta com entradas físicas (switches) e saídas (displays de 7 segmentos e LEDs). A abordagem de design modular, iniciada com as portas lógicas de 1 bit, foi consistentemente aplicada em todos os estágios. Partimos de componentes simples para construir blocos mais complexos como somadores e, finalmente, a ULA completa. Esta estratégia foi crucial para gerenciar a complexidade e garantir que cada parte do sistema fosse testável e robusta antes da integração final.

## Descrição e Decisões de Projeto dos Módulos

### 1. Blocos Fundamentais: and1bit e or1bit

```
module and1bit(a, b, s);
   input a, b;
   output s;

   assign s = a & b;

endmodule
```

```
module or1bit(a, b, s);
   input a, b;
   output s;

   assign s = a | b;

endmodule
```

Como ponto de partida para todo o sistema, criamos as portas lógicas mais básicas para operações de um único bit.

*Descrição*: O módulo and1bit realiza a operação lógica AND , enquanto o or1bit realiza a operação OR entre duas entradas de 1 bit.

*Decisões de Projeto*: Para estes componentes elementares, decidimos utilizar o modelo de fluxo de dados (dataflow). A implementação foi feita com uma única linha de código usando a diretiva assign (assign s = a & b; e assign s = a | b;). Escolhemos essa abordagem por ser a forma mais direta, legível e eficiente de descrever uma lógica combinacional simples em Verilog, o que garante que a ferramenta de síntese gere o hardware mais otimizado possível.

## 2. Operadores de 32 bits: and32bits e or32bits

```
1   module and32bits(a, b, s);
2      input[31:0] a, b;
3      output[31:0] s;
4
5      genvar i;
6      generate
7         for(i = 0; i < 32; i = i + 1)
8         begin : ripple
9            andlbit op_and(a[i], b[i], s[i]);
10        end
11     endgenerate
12
13  endmodule
```

```
1   module or32bits(a, b, s);
2      input[31:0] a, b;
3      output[31:0] s;
4
5      genvar i;
6      generate
7         for(i = 0; i < 32; i = i + 1)
8         begin : ripple
9            orlbit op_or(a[i], b[i], s[i]);
10        end
11     endgenerate
12
13  endmodule
```

Com as portas lógicas de 1 bit prontas, o próximo passo foi estender essa funcionalidade para operar em barramentos de 32 bits.

*Descrição*: Os módulos and32bits e or32bits realizam as operações lógicas AND e OR, bit a bit, em duas entradas de 32 bits.

*Decisões de Projeto*: Aqui, tomamos uma importante decisão de design: em vez de usar uma simples atribuição de fluxo de dados (como assign s = a & b;), optamos por uma abordagem estrutural. Decidimos construir o operador de 32 bits instanciando 32 vezes os nossos módulos de 1 bit. Para implementar isso de forma limpa e escalável,

utilizamos um laço for dentro de um bloco generate. Essa escolha reforça os princípios de modularidade e reuso, criando uma hierarquia de projeto clara, onde um componente complexo é explicitamente construído a partir de blocos mais simples que já foram projetados e validados.

## 3. Multiplexadores: mux2por1, mux8por1_1bit e mux8por1_32bits

Os multiplexadores são peças centrais em nosso projeto, responsáveis pela seleção de dados em pontos críticos, como na Unidade Lógica e Aritmética (ULA). Desenvolvemos uma família de multiplexadores de forma hierárquica.

➢ **mux2por1**:

```
1  module mux2por1(I0, I1, sel, s);
2     input I0, I1, sel;
3     output s;
4
5     assign s = (~sel & I0) | (sel & I1);
6
7  endmodule
```

*Descrição*: Este é o nosso multiplexador mais básico, que seleciona uma de duas entradas de 1 bit com base em um sinal de controle.

*Decisões de Projeto*: Implementamos este bloco utilizando sua equação booleana fundamental: assign s = (~sel & I0) | (sel & I1);. Esta abordagem de fluxo de dados foi escolhida por ser a representação mais clássica e eficiente do componente.

➢ **mux8por1_1bit**:

```
1   module mux8por1_1bit(I0, I1, I2, I3, I4, I5, I6, I7, sel, s);
2      input I0, I1, I2, I3, I4, I5, I6, I7;
3      input [2:0] sel;
4      output s;
5
6      wire temp_mux1, temp_mux2, temp_mux3, temp_mux4, temp_mux5, temp_mux6;
7
8      // mux2por1(I0, I1, sel, s)
9      mux2por1 mux1 (I0, I1, sel[0], temp_mux1);
10     mux2por1 mux2 (I2, I3, sel[0], temp_mux2);
11     mux2por1 mux3 (I4, I5, sel[0], temp_mux3);
12     mux2por1 mux4 (I6, I7, sel[0], temp_mux4);
13
14     mux2por1 mux5 (temp_mux1, temp_mux2, sel[1], temp_mux5);
15     mux2por1 mux6 (temp_mux3, temp_mux4, sel[1], temp_mux6);
16
17     mux2por1 mux7 (temp_mux5, temp_mux6, sel[2], s);
18
19  endmodule
```

*Descrição*: Este módulo seleciona uma de oito entradas de 1 bit, utilizando um seletor de 3 bits (sel[2:0]).

*Decisões de Projeto*: Em vez de uma abordagem comportamental com um comando case, optamos por construir o MUX de 8 para 1 estruturalmente, a partir do nosso mux2por1. Criamos uma árvore de multiplexadores de dois níveis: a primeira camada de MUXes é controlada por sel[0] , a segunda por sel[1], e a última por sel[2]. Essa decisão foi fundamental para demonstrar como um componente complexo pode ser montado a partir de blocos menores e reutilizáveis, uma prática comum e eficiente no design de circuitos digitais.

➢ **mux8por1_32bits**:

```
1   module mux8por1_32bits(res_and, res_or, res_soma, res_slt, ULAop, s);
2      input [31:0] res_and, res_or, res_soma, res_slt;
3      input [2:0] ULAop;
4      output[31:0] s;
5
6      // I0 = and; I1 = or; I2 = add; I6 = subtract; I7 = slt
7
8      genvar i;
9      generate
10        for(i = 0; i < 32; i = i + 1)
11        begin : ripple
12           // mux8por1_1bit(        I0,        I1,          I2,   I3,   I4,   I5,          I6,         I7,   sel, s)
13           mux8por1_1bit op(res_and[i], res_or[i], res_soma[i], 1'b0, 1'b0, 1'b0, res_soma[i], res_slt[i], ULAop, s[i]);
14        end
15     endgenerate
16
17  endmodule
```

*Descrição*: Este é o multiplexador principal na saída da nossa ULA. Ele seleciona o resultado de 32 bits de uma das operações disponíveis (AND, OR, Soma, etc.) com base no sinal de controle ULAop.

*Decisões de Projeto*: Seguindo a mesma filosofia de design dos operadores and32bits e or32bits, projetamos este componente de forma estrutural e escalável. Utilizamos um bloco generate para instanciar 32 vezes o nosso módulo mux8por1_1bit, onde cada instância é responsável por selecionar um bit do barramento de 32 bits. Essa decisão de projeto não só reutiliza um componente já validado (mux8por1_1bit), mas também mantém a consistência e a modularidade da nossa arquitetura.

## 4. Módulos Auxiliares e de Display

Para complementar a funcionalidade do sistema, criamos módulos para tarefas específicas como extensão de sinal e exibição de dados.

➢ **extensor_sinal**:

```
1  module extensor_sinal(a, s);
2     input [15:0] a;
3     output [31:0] s;
4
5     assign s = {{16{a[15]}}, a};
6
7  endmodule
```

*Descrição*: Este módulo converte um número de 16 bits em um de 32 bits, preservando o seu valor (extensão de sinal).

*Decisões de Projeto*: Decidimos usar os operadores de replicação e concatenação nativos do Verilog: assign s = {{16{a[15]}}, a};. Esta abordagem foi escolhida por sua incrível concisão e clareza. Ela descreve o hardware de forma precisa, replicando o bit mais significativo (a[15]) pelas 16 novas posições, sendo muito mais eficiente do que uma implementação comportamental.

➢ **hex_display**:

```
1   module hex_display(in,out);
2
3   input [3:0] in;
4   output reg [6:0] out;
5
6   always @(in)
7      begin
8         case (in)
9         4'b0000: out = 7'b1000000;
10        4'b0001: out = 7'b1111001;
11        4'b0010: out = 7'b0100100;
12        4'b0011: out = 7'b0110000;
13        4'b0100: out = 7'b0011001;
14        4'b0101: out = 7'b0010010;
15        4'b0110: out = 7'b0000010;
16        4'b0111: out = 7'b1111000;
17        4'b1000: out = 7'b0000000;
18        4'b1001: out = 7'b0010000;
19        default: out = 7'b1111111;
20        endcase
21     end
22
23  endmodule
```

*Descrição*: É um decodificador que converte um valor de 4 bits para os 7 sinais necessários para acender um display de 7 segmentos, mostrando o valor em hexadecimal.

*Decisões de Projeto*: Para uma função de mapeamento como esta, a escolha mais lógica foi uma implementação comportamental. Utilizamos um bloco always sensível à entrada e uma instrução case para mapear diretamente cada valor binário de 4 bits ao seu respectivo código de 7 segmentos. Esta abordagem é muito mais legível, intuitiva e menos suscetível a erros do que tentar derivar e escrever as equações lógicas para cada um dos 7 segmentos.

➢ **display_sinal**:

```
1   module display_sinal(bit_sinal, seg);
2      input bit_sinal;
3      output [6:0] seg;
4
5      assign seg = {~bit_sinal, 6'b111111};
6
7   endmodule
```

*Descrição*: Um módulo simples projetado para mostrar um sinal de negativo (-) em um display de 7 segmentos quando a entrada bit_sinal é '1'.

*Decisões de Projeto*: Optamos por uma implementação de fluxo de dados muito compacta: assign seg = {~bit_sinal, 6'b111111};. Se bit_sinal for '1' (negativo), a saída se torna 7'b0111111, que em um display de anodo comum acende apenas o segmento central (g), formando o sinal '-'. Caso contrário, todos os segmentos são apagados. Escolhemos esta solução por sua elegância e eficiência, resolvendo o problema com uma única linha de código.

## 5. Módulos Aritméticos: somador1bit, somador32bits e somador6bits

A base de todas as operações aritméticas do nosso sistema é o somador. Seguindo nossa filosofia de design, começamos pelo bloco mais fundamental.

➢ **somador1bit**:

```
1  module somador1bit(a, b, cin, s, cout);
2     input a, b, cin;
3     output s, cout;
4
5     assign s = a^b^cin;
6     assign cout = (a&b) | (a&cin) | (b&cin);
7
8  endmodule
```

*Descrição*: Realiza a soma de três bits de entrada (**a**, **b**, **cin**) e produz um bit de soma (**s**) e um bit de transporte (**cout**).

*Decisões de Projeto*: Para este componente, escolhemos a implementação mais direta e eficiente: o modelo de fluxo de dados baseado nas equações lógicas fundamentais de um somador completo. A soma foi implementada com uma operação XOR (**assign s = a^b^cin;**) e o transporte de saída com a sua expressão booleana padrão (**assign cout = (a&b) | (a&cin) | (b&cin);**). Esta abordagem garante uma síntese de hardware otimizada e é universalmente compreendida.

➢ **somador32bits**:

```
module somador32bits(a, b, cin, s, cout, overflow);
   input [31:0] a, b;
   input cin;
   output[31:0] s;
   output cout, overflow;

   wire [32:0] carry;

   assign carry[0] = cin;
   assign cout = carry[32];
   assign overflow = carry[32] ^ carry[31];

   genvar i;
   generate
      for(i = 0; i < 32; i = i + 1)
      begin : ripple
         //      somador1bit(a   , b       , cin     , s   , cout);
         somador1bit op_soma(a[i], b[i]^cin, carry[i], s[i], carry[i+1]);
      end
   endgenerate

endmodule
```

*Descrição*: Este módulo realiza a soma ou a subtração de dois números de 32 bits.

*Decisões de Projeto*: A decisão de projeto mais importante aqui foi a implementação de um somador/subtrator unificado. Para a soma, instanciamos 32 módulos **somador1bit** em cascata, criando um ripple-carry adder, utilizando um laço `for` dentro de um bloco `generate` para manter o código limpo e escalável. A funcionalidade de subtração é obtida através da lógica do complemento de dois. Em vez de adicionar um inversor separado, tomamos a decisão de controlar a inversão da segunda entrada (**b**) diretamente na instanciação do **somador1bit**, usando **b[i]^cin**. Quando **cin** (conectado ao **ULAop[2]**) é '1', a entrada **b** é invertida, e o **cin** inicial '1' completa a operação de complemento de dois, realizando **a - b**. As saídas de **overflow** e **cout** foram calculadas com base nos bits de transporte mais significativos (**carry[32]** e **carry[31]**).

➢ **somador6bits**:

```
1   module somador6bits(b, cin, s);
2      input [5:0] b;
3      input cin;
4      output [5:0] s;
5
6      // o somador sera usado somente para pegar o módulo de um número
7      wire [6:0] carry;
8
9      assign carry[0] = cin;
10
11     genvar i;
12     generate
13        for(i = 0; i < 6; i = i + 1)
14        begin : ripple
15           //      somadorlbit(a   , b       , cin     , s   , cout);
16           somadorlbit inverte(1'b0, b[i]^cin, carry[i], s[i], carry[i+1]);
17        end
18     endgenerate
19
20  endmodule
```

*Descrição*: Um somador especializado de 6 bits, projetado para ser usado no cálculo do valor absoluto.

*Decisões de Projeto*: Este módulo segue exatamente a mesma arquitetura do **somador32bits**, mas em uma escala menor. A decisão de criar um somador separado de 6 bits em vez de reutilizar o de 32 bits foi baseada na otimização de recursos para a tarefa específica de calcular o valor absoluto do resultado, que sabíamos que caberia em 6 bits.

## 6. Unidade Lógica e Aritmética: ULA32bits

```
module ULA32bits(a, b, ULAop, zero, s, overflow);
   input [31:0] a, b;
   input [2:0] ULAop;
   output [31:0] s;
   output overflow, zero;

   wire [31:0] res_and, res_or, res_soma, res_slt;
   wire cout;

   // and32bits(a, b, s);
   and32bits op_and(a, b, res_and);

   // or32bits(a, b, s);
   or32bits op_or(a, b, res_or);

   //    somador32bits  (a, b, cin,          soma, cout, overflow);
   somador32bits op_soma(a, b, ULAop[2], res_soma, cout, overflow);

   assign res_slt = {31'b0, overflow ^ res_soma[31]};

   // mux8por1_32bits(res_and, res_or, res_soma, res_slt, ULAop, s);
   mux8por1_32bits op(res_and, res_or, res_soma, res_slt, ULAop, s);

   assign zero = ~|s;

endmodule
```

*Descrição:* Este é o coração do nosso projeto. A **ULA32bits** integra operações lógicas e aritméticas em um único módulo, selecionando uma delas com base no sinal de controle **ULAop**. Além do resultado principal, o módulo também gera as flags de status zero e overflow, essenciais para o controle de fluxo em um processador.

*Decisões de Projeto:* Nossa principal decisão de projeto continua sendo estrutural, porém com otimizações significativas. Instanciamos os blocos para as operações lógicas (**and32bits**, **or32bits**) e um único bloco aritmético (**somador32bits**) que agora é responsável tanto pela soma quanto pela subtração. A seleção entre soma e subtração é feita de forma eficiente pelo bit **ULAop[2]**, que é conectado diretamente ao carry-in do somador.

A mudança mais notável foi na implementação do slt (set on less than). Em vez de um módulo dedicado, o resultado é agora gerado de forma combinacional (**assign res_slt = {31'b0, overflow ^ res_soma[31]};**), derivando o bit de "menor que"

diretamente dos sinais de overflow e do bit de sinal da operação de subtração, uma abordagem muito mais elegante e otimizada.

As saídas de todas essas operações são então roteadas para um multiplexador (**mux8por1_32bits**), que, sob o controle do **ULAop**, seleciona o resultado final a ser enviado para a saída s. A implementação da flag zero foi mantida (**assign zero = ~|s;**), pois a redução NOR continua sendo a maneira mais performática de se verificar se todos os bits de um barramento são zero em Verilog.

## 7. Módulos de Pós-processamento e Interface: valor_absoluto e ULA_top

➢ **valor_absoluto**:

```
1   module valor_absoluto(s, s_absoluto, bit_sinal);
2         input [5:0] s;
3         output [5:0] s_absoluto;
4         output bit_sinal;
5
6         assign bit_sinal = s[5];
7
8         // somador6bits (b, cin, s);
9         somador6bits abs(s, bit_sinal, s_absoluto);
10
11  endmodule
```

*Descrição*: Recebe um número de 6 bits e retorna seu valor absoluto e um bit de sinal.

*Decisões de Projeto*: A lógica se baseia na detecção do sinal do número através do seu bit mais significativo (**assign bit_sinal = s[5];**). Se o número for negativo (**bit_sinal = 1**), ele precisa ser convertido usando o complemento de dois. Decidimos reutilizar o **somador6bits** para essa tarefa. Passamos o próprio **bit_sinal** como **cin** para o somador. Se o número for positivo (**bit_sinal = 0**), a operação se torna **s XOR 0** com **cin = 0**, o que não altera o valor. Se for negativo (**bit_sinal = 1**), a operação se torna **s XOR 1** com **cin = 1**, que é exatamente a operação de complemento de dois (inverte e soma 1). Esta foi uma decisão de design elegante que evitou o uso de multiplexadores ou lógica condicional adicional.

➢ **ULA_top**:

```
module ULA_top(SW, HEX0, HEX1, HEX2, HEX3, HEX4, HEX5, HEX6, HEX7, LEDR0, LEDG0);
   input [17:0] SW;
   output [6:0] HEX0, HEX1, HEX2, HEX3, HEX4, HEX5, HEX6, HEX7;
   output LEDR0;
   output LEDG0;

   wire [3:0] a_sw, b_sw;
   wire [2:0] op;

   assign a_sw = SW[17:14];
   assign b_sw = SW[10:7];
   assign op = SW[2:0];

   wire [31:0] a_32, b_32;

   // extensor_sinal(a, s);
   extensor_sinal a_estendido({12'b0, a_sw}, a_32);
   extensor_sinal b_estendido({12'b0, b_sw}, b_32);

   wire [31:0] s_32;

   // ULA32bits (a,       b, op,    zero,    s, overflow);
   ULA32bits ula(a_32, b_32, op,  LEDG0, s_32, LEDR0);

   wire [5:0] s_6bit, s_abs;
   wire s_sinal;

   assign s_6bit = s_32[5:0];

            // valor_absoluto     (s, s_absoluto, bit_sinal);
            valor_absoluto res_abs(s_6bit, s_abs, s_sinal);

            // mostra o valor absoluto do resultado
            wire [3:0] s_unidade, s_dezena;
            assign s_unidade = s_abs % 4'b1010;
            assign s_dezena = s_abs / 4'b1010;

            // display_sinal       (bit_sinal, seg);
            display_sinal sinal_neg(s_sinal, HEX2);
            hex_display s_uni(s_unidade, HEX0);
            hex_display s_dez(s_dezena, HEX1);

            wire [3:0] a_unidade, a_dezena, b_unidade, b_dezena;
            assign a_unidade = a_sw % 4'b1010;
            assign a_dezena = a_sw / 4'b1010;
            assign b_unidade = b_sw % 4'b1010;
            assign b_dezena = b_sw / 4'b1010;

            // hex_display(in,out);
            hex_display a_dez(a_dezena, HEX7);
            hex_display a_uni(a_unidade, HEX6);
            hex_display b_dez(b_dezena, HEX5);
            hex_display b_uni(b_unidade, HEX4);

            // apafa o display que nao sera usado
            assign HEX3 = 7'b1111111;

         endmodule
```

*Descrição*: O módulo de mais alto nível, que integra a ULA com o mundo exterior. Ele lê as entradas dos switches do kit de desenvolvimento, as formata, as envia para a ULA e, em seguida, processa e exibe os resultados nos displays de 7 segmentos.

*Decisões de Projeto*:

1. Entrada: Decidimos mapear os switches para as entradas **a** (4 bits), **b** (4 bits) e **op** (3 bits). Como nossa ULA opera com 32 bits, utilizamos nosso módulo **extensor_sinal** para converter as entradas de 4 bits para 32 bits antes de passá-las para a ULA.

2. Execução: A ULA é instanciada e conectada diretamente às entradas processadas e às saídas de LED para as flags de **zero** e **overflow**.

3. Saída e Exibição: Para a exibição, tomamos a decisão de mostrar o valor absoluto do resultado, para simplificar a visualização em displays de 7 segmentos. Para isso, pegamos os 6 bits menos significativos do resultado da ULA, passamos pelo nosso módulo **valor_absoluto** e, em seguida, implementamos uma lógica de divisão e módulo por 10 para separar o valor em dezenas e unidades. Cada dígito (das entradas e do resultado) é então enviado para uma instância do nosso decodificador **hex_display**. Além disso, instanciamos o **display_sinal** para mostrar um sinal de negativo (-) quando o resultado original era negativo. Por fim, apagamos os displays não utilizados para uma interface mais limpa.

## 8. Memória de Dados: memoriaDados

```
module memoriaDados(address, memWrite, memRead, clock, writeData, readData);
   // 2^6 = 64 posicoes de memoria
   input [5:0] address;
   input memWrite, memRead, clock;
   input [31:0] writeData;
   // é declarado cmo reg porque armazena valor dentro do always
   output reg [31:0] readData;

   // cada posicao guarda 8 bits (1 byte); bloco de memória de 64 bytes = 512 bits;
   reg [7:0] memory [0:63];

   // quando o clock estiver na borda de subida, faz a verificação dos blocos condicionais
   always @(posedge clock)
   begin
      if(memWrite)
      begin
         // escreve na memória 4 bytes (32 bits); 1o byte em address, 2o byte em address + 1 e assmi por diante
         memory[address] <= writeData[7:0];
         memory[address + 1] <= writeData[15:8];
         memory[address + 2] <= writeData[23:16];
         memory[address + 3] <= writeData[31:24];
      end

      if(memRead)
      begin
         // lê da memória 4 bytes (32 bits); na ordem contraria à ordem lida como é feito no mips
         readData <= {memory[address + 3], memory[address + 2], memory[address + 1], memory[address]};
      end
   end
endmodule
```

*Descrição*: O módulo implementa uma memória de dados síncrona com capacidade de 64 bytes capaz de ler e escrever palavras completas de 32 bits. Ele simula o

comportamento da memória principal em uma arquitetura de computador.

*Decisões de Projeto*:

1. Armazenamento: Declaramos a memória como um array de vetores de 8 bits: reg [7:0] **memory** [0:63], fazendo com que a memória tenha 64 bytes = 512 bits. Cada endereço aponta para um único byte.

2. Endereço: Alteramos o input do endereço **address** fornecido no roteiro de 5 bits ($2^5$) para 6 bits ($2^6$) para endereçar 64 posições, garantindo que todas as posições de memória fossem acessíveis.

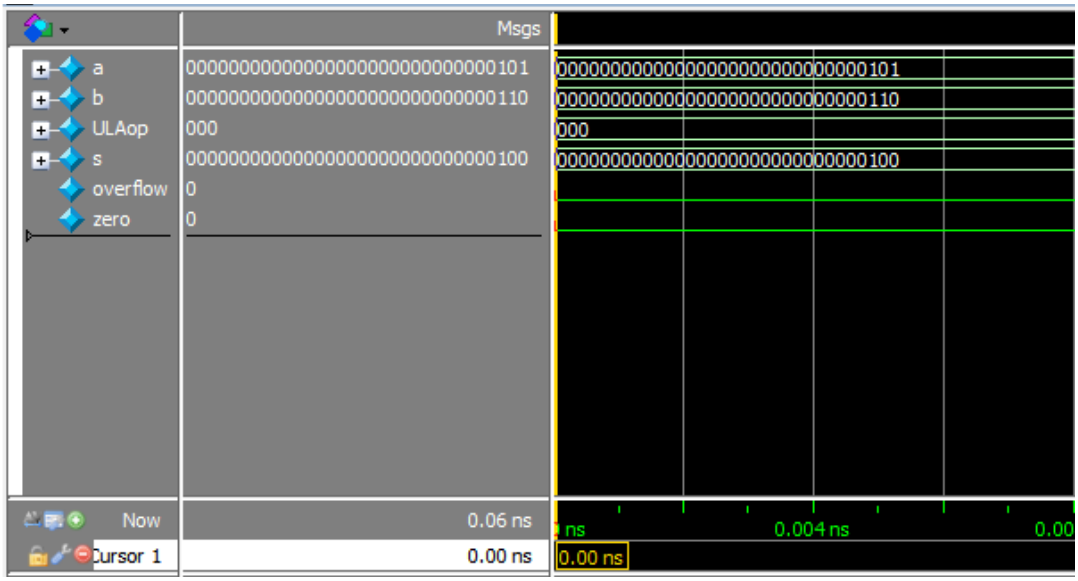
3. Sincronismo: Todas as operações de leitura e escrita foram implementadas dentro de um bloco **always** @(posedge clock). Garantindo que o estado da memória só mude na borda de subida do clock.

4. Escrita (**memWrite**): Implementamos a escrita de uma palavra dividindo o **writeData** de 32 bits em quatro bytes separados e escrevendo-os em quatro endereços de memória consecutivos (**memory**[**address**] até **memory**[**address+3**]).

5. Leitura (**memRead**): Para a leitura, fizemos o processo inverso. Lemos os quatro bytes de quatro endereços consecutivos.

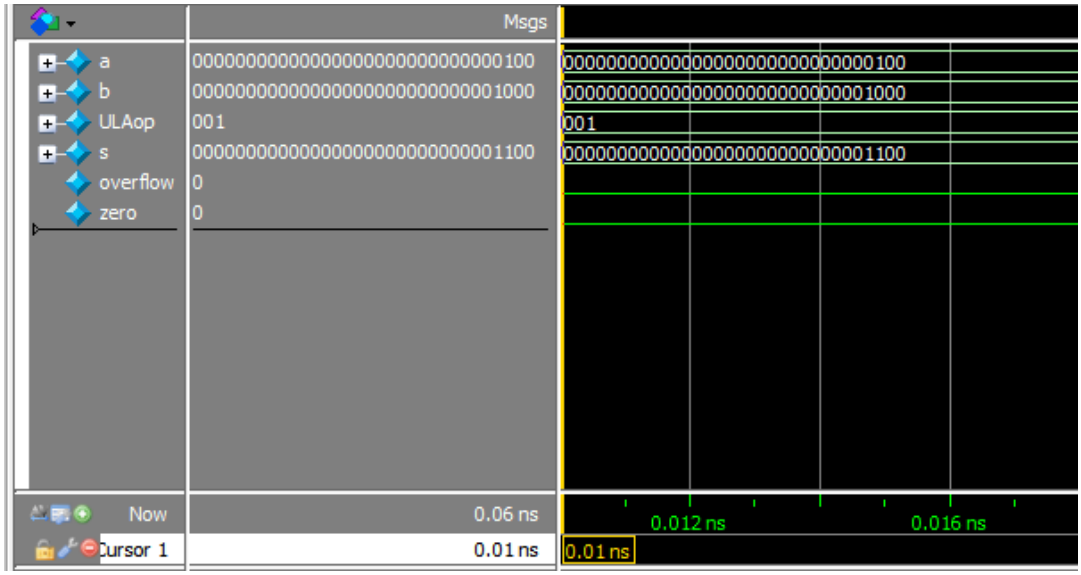
## 3. Simulações

- **Operação AND:**



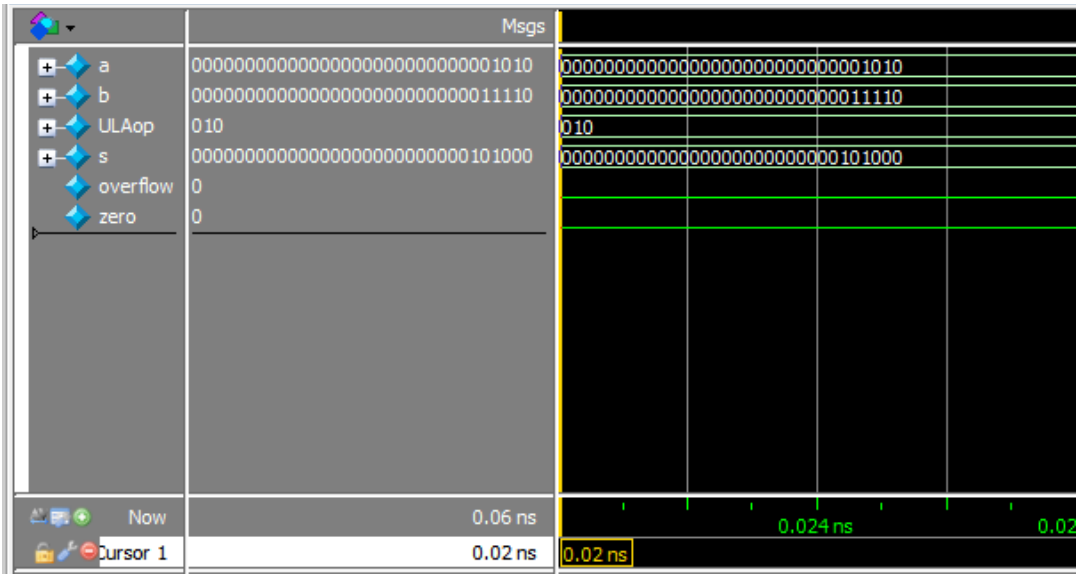
No primeiro teste da simulação, escolhemos os valores a = 101 (5 em binário), b = 110 (6 em binário) e ULAop = 000, que seleciona, no mux, a operação lógica AND. Dado que a porta lógica AND só é 1 quando os dois operandos são 1: 101 & 110 = 100 (4 em binário).

- **Operação OR:**



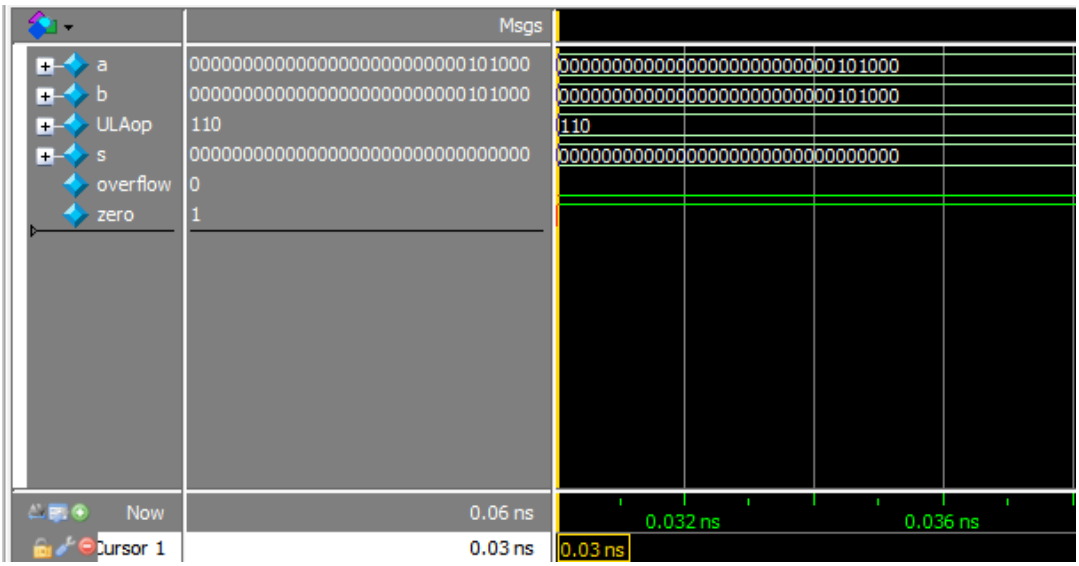

No segundo teste da simulação, escolhemos os valores a = 100 (4 em binário), b = 1000 (8 em binário) e ULAop = 001, que seleciona, no mux, a operação lógica OR. Dado que a porta lógica OR é 1 sempre que um dos operandos é 1: 100 | 1000 = 1100 (12 em binário).

- **Operação ADD:**



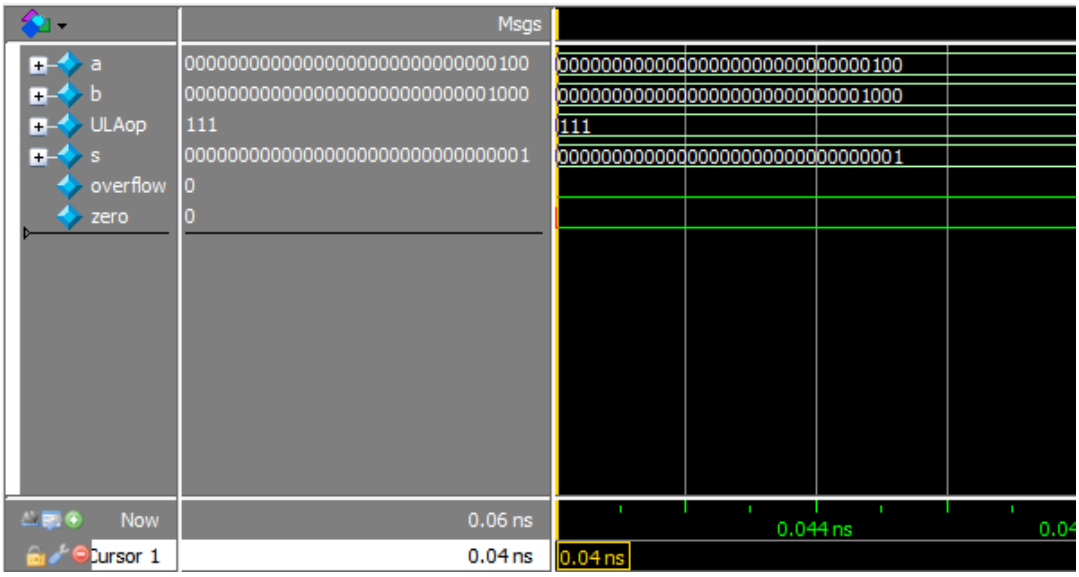

No terceiro teste da simulação, escolhemos os valores a = 1010 (10 em binário), b = 11110 (30 em binário) e ULAop = 010, que seleciona, no mux, a operação ADD. A operação realizada é: 1010 + 11110 = 101000 (40 em binário).

- **Operação SUB:**



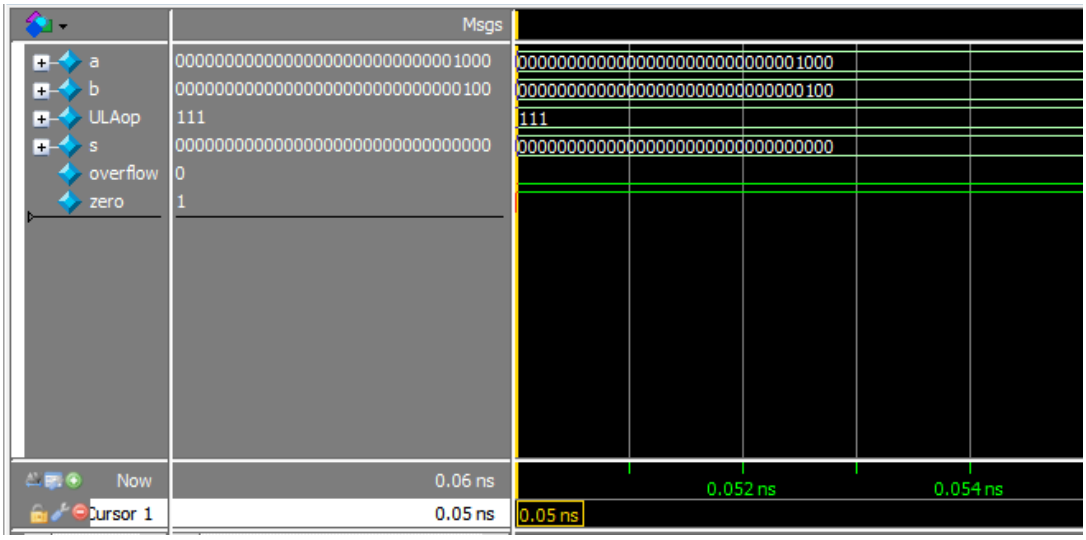

No quarto teste da simulação, escolhemos os valores a = b = 101000 (40 em binário) e ULAop = 110, que seleciona, no mux, a operação SUB. A operação realizada é: 101000 - 101000 = 0. Tal operação zera a saída s e faz com que a flag do zero se torne verdadeira (1), indicando que a saída está zerada.

- **Operação SLT (verdadeiro):**




No quinto e último teste da simulação, escolhemos os valores a = 100 (4 em binário), b = 1000 (8 em binário) e ULAop = 111, que seleciona, no mux, a operação SLT. A operação realizada é: 100 < 1000, que retorna 1 (verdadeiro) pois 4 é menor que 8.

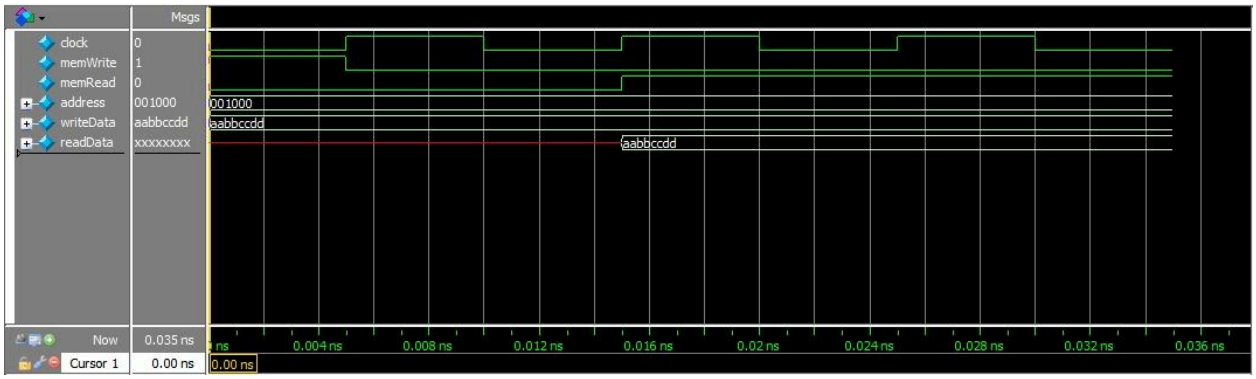
- **Operação SLT (falso):**

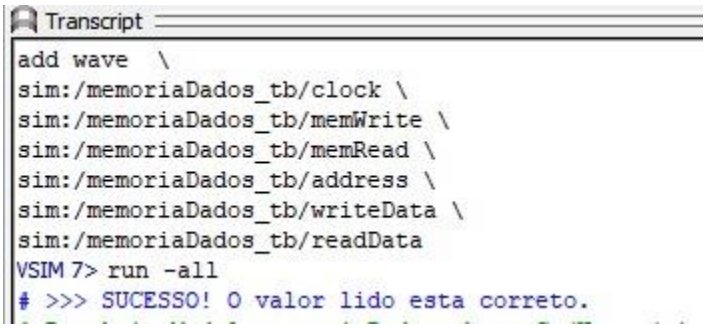



O segundo teste do SLT faz a = 1000 (8 em binário) e b = 100 (4 em binário). A operação realizada 1000 < 100 retorna 0, pois 8 não é menor que 4, o que faz com que

a saída s zere e a flag da saída zero se torne verdadeira (1).

- **Memória de Dados:**




Na inicialização, seta memRead em 0 e memWrite em 1, armazenando o dado, que é um número de 32 bits em hexadecimal, aabbccdd, (2864434397 em decimal) no endereço de memória de address, 001000 (8 em decimal). Depois, seta memRead em 1 e memWrite em 0 e, quando está em clock de subida, o número é lido da memória para readData. Depois, para fazer a confirmação, utilizamos um bloco condicional no testbench para verificar se o valor lido é igual ao valor digitado, imprimindo o resultado no bloco de texto do simulador (mas isso também pode ser verificado avaliando o gráfico, pois readData possui o mesmo valor de writeData após a segunda subida de clock).